FAIRLANE · PHASE II REHEARSAL GUIDE · FILE 3 OF 3

Crash Test

A worker dies mid-job, the scheduler recovers, and no work is lost

Team Orchestrators · Fairlane: A Fair and Fault-Tolerant Scheduler for Distributed ML Training Workloads

WHO IS WHO

- VANSH'S PC = the central SCHEDULER folder: ~/Code/GPU_Orchestrator
- SURYANSH-VM = the WORKER AGENT folder: ~/Distributed_GPU_Workload_Orchestrator/gpu-orchestrator/worker-agent

WHAT THIS SHOWS

A worker machine dies while running a job. The scheduler notices the missing heartbeats, marks the worker OFFLINE and puts the job back in the queue. When the worker comes back, it re-registers by itself and the job runs again to completion.

HOW IT WORKS

in case the evaluator asks

- The agent sends a heartbeat every 10 seconds.
- The scheduler's reaper checks regularly; a worker silent for more than 30 seconds is marked OFFLINE.
- Its running job is marked FAILED with reason `worker lost (heartbeat timeout)` and the JOB goes back to QUEUED.
- When the worker registers again, the scheduler sends the job again.

HOW TO READ THIS FILE

- Lines under `>> PASTE:` are commands. Copy them exactly, one line at a time, and press Enter after each.
- Replace **<VANSH_IP>** and **<VM_IP>** with the real numbers (no < >).
- `EXPECT:` tells you what a correct result looks like.
- `SAY:` is the line to tell the evaluator.

BEFORE YOU START

- File 2 (Demo) Parts 0-2 are done: scheduler running in V1 on Vansh's PC, agent running in S1 on the VM, both on the same hotspot.
- Terminals: Vansh's PC: V1 (server) + V2 (commands)

```
suryansh-vm: S1 (agent)  + S2 (for the
"kill" command)
```

IMPORTANT DO NOT use Ctrl+C on the agent to simulate the crash. Ctrl+C also kills the job and the agent reports it as a normal job failure (`exit code -2`), so the job is NOT re-queued. Use the kill command below - it stops the agent instantly, exactly like a real crash.

Total Time about 3 minutes

PART 0 Prepare a Longer Job (Suryansh-vm, Once, Before the Evaluator)

Normal demo jobs finish in 20 seconds - too fast to crash in the middle. This creates a job that runs for 60 seconds.

0.1 Create the long job script

PASTE suryansh-vm S2

```
mkdir -p ~/fairlane_jobs/users/jack/long-job
printf 'import time\nprint("long job started", flush=True)\ntime.sleep(60)\nprint("done")\n' >
~/fairlane_jobs/users/jack/long-job/train.py
cat ~/fairlane_jobs/users/jack/long-job/train.py
```

EXPECT
the 4-line Python script is printed.

PART 1 Start the Job

1 Check the worker is online

PASTE Vansh's PC V2

```
cd ~/Code/GPU_Orchestrator
uv run python -m scripts.status
```

EXPECT under WORKERS
4 suryansh-vm ONLINE

2 Submit the long job

PASTE Vansh's PC V2

```
uv run python -m scripts.submit Jack long-job 3000
```

EXPECT
`submitted job N (long-job, 3000 MiB) for Jack`

Note the job number N _____

3 Confirm it is RUNNING (within ~5 seconds)

PASTE Vansh's PC V2

```
uv run python -m scripts.status
```

EXPECT
N long-job 3000 MiB RUNNING

On the VM (S1) the agent shows the job has started.

SAY TO THE EVALUATOR
"The job is now running on Suryansh's machine. Now we simulate that machine crashing."

PART 2 Crash the Worker

4 Kill the agent instantly (do it while the job is RUNNING)

PASTE suryansh-vm S2

```
pkill -9 -f "uvicorn agent:app"
```

EXPECT
S1 suddenly returns to the prompt (it may print `Killed`).

The agent had NO chance to tell the scheduler anything.

SAY TO THE EVALUATOR
"This is like pulling the power cable: the worker stops instantly and says nothing to the scheduler."

5 Show the worker is still ONLINE for a moment

PASTE Vansh's PC V2

```
uv run python -m scripts.status
```

EXPECT

suryansh-vm still ONLINE, long-job still RUNNING.

SAY TO THE EVALUATOR

"The scheduler cannot know yet. It only knows the heartbeats have stopped. We wait for the 30-second timeout."

6 Wait 45 seconds, then check again

PASTE Vansh's PC V2 after 45 seconds

```
uv run python -m scripts.status
```

EXPECT

```
WORKERS
  4 suryansh-vm OFFLINE
JOBS
  N long-job   3000 MiB  QUEUED          <- back in the queue, NOT failed
EXECUTIONS
  .. job N on worker 4  FAILED    worker lost (heartbeat timeout)
```

If it still says ONLINE, wait 10 more seconds and run it again.

SAY TO THE EVALUATOR

"After 30 seconds without a heartbeat, the reaper marked the worker offline. Only that ATTEMPT is marked failed - the job itself went back to the queue, so the user's work is not lost."

PART 3 The Worker Comes Back

7 Restart the agent

In S1 press the Up-arrow key to get the last command back, or paste:

PASTE suryansh-vm S1 — one single line

```
SERVER_URL=http://<VANSH_IP>:8000 AGENT_URL=http://<VM_IP>:8001 WORKER_NAME=suryansh-vm bash run_agent.sh
```

EXPECT

```
... registered with http://<VANSH_IP>:8000 as worker 4 (suryansh-vm)
and within a few seconds the long job starts again.
```

SAY TO THE EVALUATOR

"The worker re-registers on its own with the same identity, and the scheduler immediately sends it the queued job again."

8 Show it is running again

PASTE Vansh's PC V2

```
uv run python -m scripts.status
```

EXPECT

4 suryansh-vm ONLINE and N long-job ... RUNNING

9 Wait ~60 seconds, then the final check

PASTE Vansh's PC V2 — after 60 seconds

```
uv run python -m scripts.status
```

EXPECT

```
WORKERS
  4 suryansh-vm ONLINE
JOBS
  N long-job   3000 MiB  COMPLETED
EXECUTIONS (newest first)
  .. job N on worker 4  COMPLETED
  .. job N on worker 4  FAILED    worker lost (heartbeat timeout)
```

SAY TO THE EVALUATOR

"Two attempts: the first was lost with the crash, the second completed. The same job, no manual action, nothing lost."

Optional Variant: "Pull the Network Cable" (looks great live)

Same idea, but the agent keeps running and recovers WITHOUT a restart.

1. Vansh submits a long job (Step 2) and confirms RUNNING (Step 3).
2. Suryansh: in the VirtualBox window menu choose

   ```
   Devices -> Network -> uncheck "Connect Network Adapter"
   ```

3. Vansh: wait 45 s, run status → suryansh-vm OFFLINE, job QUEUED.
4. Suryansh: Devices → Network → CHECK `Connect Network Adapter`.
5. Within ~10 s, S1 shows:

   ```
   scheduler says this worker was declared offline; registering again
   registered with ... as worker 4
   ```

   and the job is sent again. Vansh's status shows ONLINE / RUNNING, then COMPLETED after ~60 s.

> **SAY TO THE EVALUATOR**
> *"When the network came back, the scheduler rejected the old heartbeat (HTTP 409) because it had already declared the worker dead. The agent understood that and re-registered by itself."*

NOTE the old copy of the job may still be running on the VM in the background and finish later. Its late report is ignored by the scheduler (that attempt is already marked failed), so results are not counted twice.

If Something Goes Wrong

WHAT YOU SEE	WHAT TO DO
Job FAILED `Script ... not found`	Part 0 was not done on the VM. Do it, then submit again (Step 2).
Job finished before you could kill it	Submit another long-job and be quicker, or change 60 to 120 in Part 0.
Job shows FAILED `exit code -2`	Ctrl+C was used instead of pkill. Submit again and use Step 4.
After 45 s the worker is still ONLINE	The agent is still alive. In S2 run: pgrep -af uvicorn → if something is listed, run Step 4 again.
Restart says `address already in use`	The old agent is still running. Run Step 4 again, then Step 7.
Restarted agent before the 30 s timeout and EXECUTIONS shows `worker restarted`	Also fine: on re-registration the scheduler re-queues the job itself. Explain: `a worker that registers again has restarted, so its old work is re-queued.`
Variant: worker never comes back ONLINE	VM may have a new IP. Run hostname -I; if it changed, restart the agent with the new VM_IP (Step 7).

Reset

To repeat the whole demo + crash test from zero, follow File 2, Part 6 (reset), then File 2 from Step 1.